Ceph (RADOS)

Data pool | Metadata pool | Index pool

Object storage interface built on top of the librados library to provide applications with a RESTful gateway to Ceph storage clusters. A Ceph Storage Cluster requires the following: at least one **Ceph Monitor** and at least one **Ceph Manager**, and at least as many **Ceph Object Storage Daemons** (**OSD**s) as there are copies of a given object stored in the Ceph cluster (for example, if three copies of a given object are stored in the Ceph cluster, then at least three OSDs must exist in that Ceph cluster).

Before Ceph clients can **directly read** from or **write** to Ceph **OSDs**, they must connect to a Ceph Monitor first. With a current copy of the cluster map and the **CRUSH** algorithm, a Ceph client can compute the location for any object.

Services

Services:
- OSDs
- Monitors
- Managers
- MDSes
- RGWs

OSD

Object Storage Daemon

Storage nodes - store data as objects and handle all interaction with the physical drive (or file system)

Responsible for **replication** and **recovery**

Adding:
- Provision disk
- Install OSD
- Add to CRUSH map
- Mark in

Removing:
- Mark out
- Wait for Placement Groups rebalance
- Remove from CRUSH map

- Stop daemon

Monitor

Ceph Monitors write all changes in the monitor services to a single **Paxos** instance, and Paxos writes the changes to a key-value store for **strong consistency**. Ceph Monitors can query the most recent version of the cluster map during synchronization operations. Ceph Monitors leverage the key-value store's snapshots and iterators, using the **rocksdb** database, to perform store-wide synchronization.
Due to the nature of Paxos, Ceph requires a majority of monitors running to establish a quorum, thus establishing consensus.
Odd number of monitors - more resilient.
Single source of truth for the cluster map.
Ceph is able to **operate only if a majority of monitors are active** and able to communicate with each other.

3 monitors min, 5 best advised

Best run not on same host as OSD - fsync issues with the kernel might weaken performance

Monitors - maintains **master** maps of the cluster state:
- Monitor map - fixed addresses for monitors for discovery and communication with quorum
- Manager map - status and capabilities of the Ceph Managers. **Which is active**
- OSD map - status and membership of OSDs
- MDS map - status and filesystem metadata
- CRUSH map - determines which OSDs should store an object's data based on data location and the cluster's topology
- PG - placement group - where an object's data and replicas are physically stored

Manager

Administrative interface for cluster

Managers - keeping track of runtime **metrics** and the current state of the Ceph cluster:
- storage utilization
- performance metrics
- system load

host **python**-based modules to **manage** and **expose** Ceph **cluster information**, including a web Dashboard and **REST API**

2 normally **required** for high availability, but **better 3**. **1 Active, all others Standby**

If active fails, one of Standby is selected -> Active

Polls the OSDs and Monitors: **health, metrics, performance** supports integration of Prometheus + Grafana

Modules:

- Dashboard
- Balancer
- Progress - progress of long-running operations
    - recovery
    - rebalancing
    - OSD removals/additions
- Telemetry - anonymous cluster usage -> Ceph devs
- Prometheus

MDS

MDSes - Metadata servers:

- Directory structure/namespaces - **Dynamic Subtree Partitioning** to distribute
- File attributes - default POSIX
- File layout - Data object mapping - which RADOS objects store the file data
- Extended attributes - arbitrary metadata key/value for special cases
- Client capabilities - issues **capabilities** for clients

Capabilities - **tokens** for **permissions** (r/w) and size. Used for caching permissions - client does **1 request** to MDS **per file** processed

Granular - categories:

- G - get - file size, mode, UID, GID
- X - extend - cache and modify Extended attributes
- D - dir - contents of directory
- W - write - write to file + data layout
- R - read - read file

RGW

RADOS Gateway - bridge between HTTP clients (S3/Swift) and the raw Ceph storage cluster (RADOS)

Based on **Beast**, Stateless

Need external balancer

Multi-Site Replication:

- Realm -> zone groups -> zones - RGW + RADOS (Ceph)

RADOS pools

Types:
- Logical - what is stored, multiple instances allowed (typically data):
  - Index
  - Data
  - Metadata
- Physical - how is stored:
  - Replicated - full objects in all replicas - hot storage
  - Erasure coded - cold storage

Metadata - user and bucket records (bucket entry points, user info)
Index - bucket index (objects metadata list: names, sizes, timestamps) - Bucket Index Sharding
Data - actual object contents. Can use Erasure Coding for high space efficiency, but adds latency

Bucket index sharing - bucket -> shards - separate RADOS objects
Key **hash** based shard index

Resharding - when avg amount of objects > 100,000 objects per shard
Usually grows 1 -> 16 -> 32 -> …. -> max_shards
Read - mostly unaffected
Write - can be blocked while writing metadata
Available in Luminous and later
Manual resharding is only available **offline**

Erasure coding

Mathematical algorithm, split data into chunks data (K) + parity (M)

Requires K + M OSDs, but can **tolerate loss** of any up to **M**

Slower read/write, but way **better space/safety** efficiency

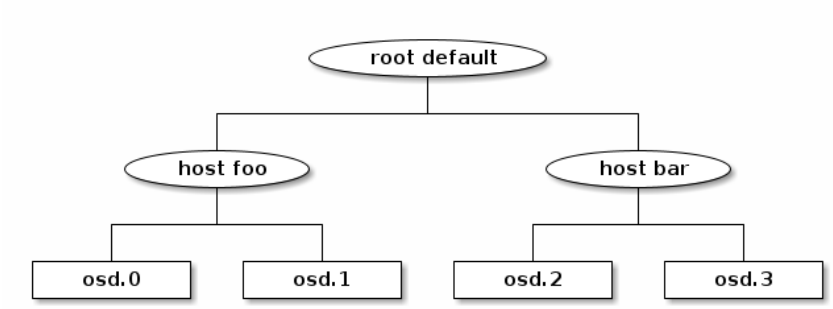
If any of data chunks is lost, object is recreated using parity chunks
If all are present object is simply “assembled”

CRUSH

CRUSH Map:

- Devices - OSDs
- Buckets - hardware hierarchy - hosts, racks, rows, data centers
- Rules - how data is replicated or erasure-coded across the topology

For example, to address the possibility of concurrent failures, it might be desirable to ensure that data replicas are on devices that reside in or rely upon different shelves, racks, power supplies, controllers, or physical locations.



Rules:
- Location
- Replication factor
- Failure domain
- Erasure coding rules
- **Logical pool** link (1-1)

Ceph CSI

Ceph <-> k8s interface: provisioning + attachment
Two drivers:
- **RBD**: block storage (rbd.csi.ceph.com)
- **CephFS**: file storage (cephfs.csi.ceph.com)

Deployed as:
- **Controller plugin**: Deployment with the provisioner, attacher, resizer and snapshotter sidecars
- **Node plugin**: DaemonSet that maps and mounts volumes on each node. Needs privileged mode and hostNetwork, and mounts /dev and /sys

Needs:
- clusterID = the Ceph fsid
- Monitor addresses in the csi-config ConfigMap
- A cephx user and key, stored in Secrets

Secrets:

- provisioner-secret: creates and deletes images or subvolumes
- node-stage-secret: maps and mounts volumes on the node

StorageClass parameters:

- clusterID, plus pool (RBD) or fsName (CephFS)
- imageFeatures: layering, a safe default for the kernel RBD client
- csi.storage.k8s.io/fstype: ext4 or xfs
- csi.storage.k8s.io/*-secret-name and *-secret-namespace

Access modes:

- RBD with Filesystem volumeMode: RWO
- RBD with Block volumeMode: RWX, used for raw devices such as KubeVirt disks
- CephFS: RWX

Features: volume expansion, VolumeSnapshotClass, clone and restore from snapshot
volumeBindingMode: WaitForFirstConsumer gives topology-aware scheduling

Gotchas:

- The node's kernel rbd client doesn't support every image feature, so the map fails. Fix: imageFeatures: layering
- The ceph-csi release must match the Ceph release. Check the compatibility matrix before upgrading either side

Rook

k8s operator that deploys and manages Ceph
CRDs:

- CephCluster
- CephBlockPool
- CephFilesystem
- CephObjectStore and CephObjectStoreUser
- CephClient and CephNFS

Operator spawns: MON, MGR, OSD (prepare job, then OSD pod), MDS, RGW and the CSI drivers
OSDs can be PVC-backed through storageClassDeviceSets, for clouds or nodes without raw disks
Auto StorageClasses (names are examples): rook-ceph-block, rook-cephfs. Buckets come from ObjectBucketClaim
Toolbox pod gives a ceph CLI (ceph -s, ceph osd tree)
Modes:

- **Internal**: Rook owns the cluster

- **External**: connects k8s to an existing Ceph cluster via create-external-cluster-resources.py. Usual choice when Ceph already serves OpenStack

Upgrades: the operator rolls through MON, then MGR, then OSD
Gotcha: MON count and failure domain are set in CephCluster. Changing them later is painful, so plan them up front

OpenStack + Ceph

Ceph is the default backend for all OpenStack storage

- **Cinder**: RBD driver, pool volumes, thin-provisioned, copy-on-write clones
- **Glance**: RBD store, pool images. Set show_image_direct_url = True so a boot volume is a clone of the image and starts fast
- **Nova** ephemeral disks: libvirt images_type = rbd, pool vms. Live migration works without block migration
- **Manila**: CephFS, through the native driver or NFS-Ganesha
- **Cinder backup**: pool backups
- **RGW**: S3 and Swift API with Keystone auth, which replaces Swift
- cephx user per service: client.cinder, client.glance, client.nova
    - caps: profile rbd pool=volumes, plus pool=vms, and read-only on images

Cinder RBD image naming: volume-<cinder-uuid>
Gotcha: the RBD backend doesn't support Cinder multi-attach

OpenStack + k8s

- **Magnum** builds k8s clusters on OpenStack VMs

Two ways to get persistent storage for k8s on OpenStack-backed Ceph:

- **cinder-csi-plugin**: PVC goes through the Cinder API to RBD
  - Pro: stays inside OpenStack, so quotas, volume types, Horizon visibility and existing ops tooling apply
  - Con: an extra hop, and the Nova attach API makes it slower. RWO only, and no native RBD or CephFS features
- **ceph-csi or Rook**: PVC goes straight to Ceph
  - Pro: lower latency, CephFS RWX, native snapshots and clones, and no Cinder quotas to hit
  - Con: VMs need reachability to the Ceph public network, cephx keys sit on the nodes, and Cinder doesn't know the volumes exist

Rule of thumb:

- k8s is the main consumer: go direct to Ceph
- You need OpenStack control, quotas or billing: use Cinder CSI

Rook external mode fits well when Ceph already runs for OpenStack

Tracing a volume end to end:

- Cinder: volume-<uuid> in pool volumes

- ceph-csi: PV pvc-<uuid>, RBD image csi-vol-<uuid>
- Check the mapping: kubectl get pv <pv> -o jsonpath='{.spec.csi.volumeAttributes}'

Object storage from k8s:

- Rook: ObjectBucketClaim creates an RGW bucket, with credentials in a Secret
- OpenStack: create an RGW user through Keystone, or with radosgw-admin user create

Useful links

https://forum.proxmox.com/threads/ceph-classes-rules-and-pools.60013/
https://www.youtube.com/watch?v=O7zWbuH-XWc
https://docs.ceph.com/en/latest/rados/